



TP FIL ROUGE

Construire, spécialiser et déployer un assistant IA métier local

Ollama • Open WebUI • Fine-tuning LoRA • RAG • Docker • ESXi

**Module commun — profils Développement, Systèmes, Réseaux, DevOps et
Cybersécurité**

Objectif : partir d'un LLM local générique et livrer un service IA spécialisé, documenté, testé et accessible sur le réseau.

1. Contexte du TP

TECHCORP est une entreprise fictive qui regroupe plusieurs métiers : développeurs applicatifs, administrateurs systèmes, ingénieurs réseaux, équipes DevOps/Cloud et analystes cybersécurité. Chaque équipe possède sa propre documentation : code source, documentation API, README, procédures d'exploitation, configurations, guides de dépannage, normes internes et FAQ.

La direction souhaite expérimenter une IA interne et locale. L'assistant doit fonctionner sans dépendre d'un service d'IA externe pour les documents internes, répondre à des questions propres à un métier, citer ou exploiter une base documentaire, puis être déployé sur une VM accessible depuis un navigateur.

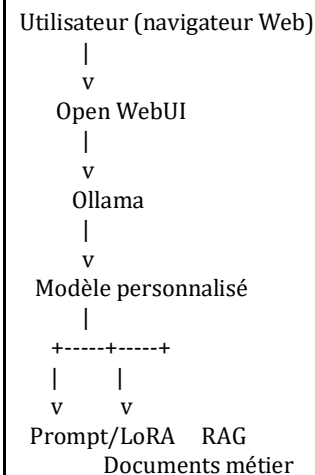
Votre mission

En partant d'un modèle local générique, vous devez construire un assistant IA adapté à votre spécialité.

Le même fil rouge s'applique à tous les profils : seul le contenu métier change entre Développement, Réseau, Système, DevOps et Cybersécurité.

À la fin, votre assistant doit fonctionner dans Open WebUI, utiliser une base RAG et être déployé sur une VM Linux hébergée sur ESXi.

Architecture cible



Compétences visées

- Comprendre LLM, Ollama, Open WebUI, Modelfile, fine-tuning et RAG.
- Installer et tester un modèle local depuis le terminal et une interface Web.
- Construire un jeu de données instruction/réponse adapté à un métier.
- Créer un assistant spécialisé sans confondre comportement, entraînement et connaissance documentaire.
- Conteneuriser la solution et la déployer sur une VM Linux.
- Tester une chaîne complète : navigateur → interface → moteur IA → modèle → documents.

2. Choisir votre profil métier

Chaque étudiant ou binôme choisit un profil. Vous gardez ce profil pendant tout le TP.

Profil	Assistant possible	Exemples de questions	Documents RAG possibles
--------	--------------------	-----------------------	-------------------------

Profil	Assistant possible	Exemples de questions	Documents RAG possibles
Développement	DevAssist-GPT	Python, FastAPI, API REST, erreurs de code, tests	README, documentation API, conventions, extraits de code, guides de tests
Systèmes	AdminLinux-GPT	services, permissions, systemd, stockage, logs	procédures Linux, runbooks, commandes, checklists
Réseaux	NetCisco-GPT	VLAN, routage, ACL, DHCP, DNS, diagnostic	schémas, configurations, procédures de dépannage
DevOps / Cloud	DevOps-GPT	Docker, Kubernetes, Git, CI/CD, déploiement	Dockerfile, compose, manifests, pipelines, procédures
Cybersécurité	CyberSOC-GPT	logs, incidents, triage, bonnes pratiques	playbooks, procédures SOC, règles internes
Virtualisation	Proxmox-GPT	VM, stockage, réseau virtuel, sauvegarde	guides Proxmox, procédures de sauvegarde et restauration

Important pour les profils Développement

Vous n'êtes pas obligés de choisir un sujet Réseau ou Linux. Votre assistant peut être centré sur Python, FastAPI, API REST, Git, tests, Docker ou CI/CD.

Dans votre dataset, vous pouvez utiliser des questions de code, des erreurs fréquentes, des explications de fonctions, des exemples de requêtes HTTP et des procédures de débogage.

Dans votre RAG, vous pouvez charger une documentation API, un README de projet, une convention de code ou une documentation d'architecture.

3. Phase 1 — Mettre en service une IA locale

Étape 1 — Installer Ollama

Téléchargement : <https://ollama.com/download>

```
ollama --version
```

Étape 2 — Télécharger et tester le modèle de base

Modèle de référence du TP : Qwen2.5 7B. Si votre machine est limitée, vous pouvez utiliser Qwen2.5 3B en secours.

```
ollama pull qwen2.5:7b
ollama run qwen2.5:7b
```

Posez au minimum trois questions : une question générale, une question de votre spécialité et une question volontairement liée à une information interne fictive que le modèle ne peut pas connaître.

- ☐ Le modèle répond dans le terminal.
- ☐ Vous identifiez au moins une réponse trop générique, incomplète ou incertaine.
- ☐ Vous notez ce que le modèle ne peut pas connaître sans documentation interne.

Étape 3 — Installer Open WebUI

Open WebUI fournit une interface de chat Web au-dessus d'Ollama.

```
docker run -d \
-p 3000:8080 \
--add-host=host.docker.internal:host-gateway \
-e OLLAMA_BASE_URL=http://host.docker.internal:11434 \
-v open-webui:/app/backend/data \
--name open-webui \
--restart always \
ghcr.io/open-webui/open-webui:main
```

Ouvrez ensuite : <http://localhost:3000>

À savoir expliquer

Ollama = moteur qui télécharge et exécute les modèles localement.
 Open WebUI = interface Web permettant de discuter avec ces modèles.
 Le modèle = le LLM qui produit réellement la réponse.

Validation Phase 1

- ☐ Qwen2.5 apparaît dans Open WebUI.
- ☐ Une conversation fonctionne depuis le navigateur.
- ☐ Vous réalisez une capture avec le modèle sélectionné.
- ☐ Vous savez expliquer la chaîne navigateur → Open WebUI → Ollama → modèle.

4. Phase 2 — Spécialiser l'assistant

Comprendre les 3 mécanismes

Mécanisme	À quoi il sert	Exemple	Quand l'utiliser
SYSTEM / Modelfile	Définir le rôle, le ton et les règles de réponse	« Tu es un assistant FastAPI... »	Toujours pour cadrer le comportement
Fine-tuning LoRA	Adapter les habitudes de réponse du modèle avec des exemples	Format de diagnostic, style de réponses, type de raisonnement	Quand on veut apprendre un comportement récurrent
RAG	Fournir des connaissances provenant de documents	README, procédure, doc API, configuration	Quand les informations sont documentaires ou évoluent souvent

Étape 4 — Construire 50 à 100 exemples instruction/réponse

Utilisez votre IA locale pour accélérer la génération, mais chaque exemple doit être relu et corrigé. Les réponses fausses créent un mauvais dataset.

20 à 30 exemples	connaissances fondamentales de votre spécialité
20 à 30 exemples	procédures pas à pas
10 à 20 exemples	diagnostic, erreur ou incident
5 à 10 exemples	cas où l'assistant doit demander des informations supplémentaires plutôt que d'inventer

Exemples — Profil Développement

Instruction : Explique la différence entre GET et POST dans une API REST.

Réponse : GET sert principalement à lire une ressource...

Instruction : Mon endpoint FastAPI retourne une erreur 422. Que vérifier ?

Réponse : Commence par vérifier le schéma Pydantic attendu, les noms des champs...

Instruction : Corrige cette fonction Python et explique l'erreur.

Réponse : ...

Exemples — Profil Réseau / Système

Instruction : Un poste obtient une adresse 169.254.x.x. Que vérifier ?

Réponse : Vérifier le serveur DHCP, le VLAN, le relais DHCP, la connectivité...

Instruction : Un service Linux ne démarre plus. Quelle démarche suivre ?

Réponse : Vérifier systemctl status, journalctl, permissions, ports, dépendances...

Format conseillé du dataset

```
[
  {
    "instruction": "Question ou tâche",
    "response": "Réponse attendue"
  }
]
```

5. Fine-tuning léger et Modelfile

Étape 5 — Fine-tuning LoRA dans Google Colab

Notebook fourni par l'enseignant :

<https://colab.research.google.com/drive/1yYAZR2kuWJlRbXpNUFM9EM9hWwNaph4R?usp=sharing>

1. Activer un runtime GPU si disponible.
2. Charger un modèle Qwen2.5 Instruct compatible avec le notebook.
3. Importer les 50 à 100 exemples validés.
4. Lancer un entraînement LoRA court.
5. Exporter l'adapter selon la méthode prévue par le notebook.
6. Conserver le nom exact du modèle de base et les paramètres utilisés.

À ne pas confondre

Un fine-tuning ne remplace pas une base documentaire.

Pour une documentation qui change souvent, privilégiez le RAG.

Le fine-tuning sert surtout à modifier des habitudes ou un format de réponse.

Étape 6 — Créer le Modelfile

Créez un fichier nommé Modelfile. Adaptez le SYSTEM à votre profil métier.

```
FROM qwen2.5:7b

SYSTEM """
Tu es DevAssist-GPT, assistant interne de TECHCORP.
Tu aides les développeurs sur Python, FastAPI, API REST, Git et Docker.
Tu expliques les termes techniques simplement avant d'aller dans le détail.
Si une information manque, tu poses une question au lieu d'inventer.
Pour les actions risquées, tu proposes d'abord une vérification.
"""

PARAMETER temperature 0.2
PARAMETER top_p 0.9
PARAMETER repeat_penalty 1.1
PARAMETER num_ctx 4096
```

Si votre export LoRA est compatible avec Ollama, ajoutez la directive ADAPTER avec le chemin de l'adapter correspondant au même modèle de base.

```
ADAPTER ./mon_adapter
```

```
ollama create devassist-gpt -f Modelfile
ollama run devassist-gpt
```

Paramètres à comprendre

temperature	créativité / variabilité ; faible valeur recommandée pour un assistant technique
top_p	diversité des mots candidats utilisés pour produire la réponse
repeat_penalty	réduit les répétitions excessives
num_ctx	taille maximale du contexte traité par le modèle

6. Ajouter une base de connaissances RAG

Étape 7 — Préparer les documents

Choisissez 3 à 8 documents non sensibles et directement utiles à votre assistant.

Profil	Documents RAG adaptés	Exemple de question de test
Développement	README, OpenAPI/Swagger exporté, conventions de code, guide de déploiement	« Quel endpoint permet de créer un ticket et quel JSON envoyer ? »
Réseau	plan IP, procédures VLAN, guide DHCP/DNS, config de labo	« Quel VLAN utilise le réseau d'administration ? »
Système	runbook systemd, sauvegarde, comptes, stockage	« Quelle procédure suivre pour restaurer le service X ? »
DevOps	Dockerfile, compose, pipeline, manifest Kubernetes	« Quelle variable doit être définie avant le déploiement ? »
Cybersécurité	playbook incident, règles, checklist	« Quelle est la première action lors d'une alerte Y ? »

Étape 8 — Créer la connaissance dans Open WebUI

- Ouvrir la section Knowledge / Connaissances.
- Créer une nouvelle collection.
- Ajouter les documents.
- Attendre l'indexation.
- Associer la collection au modèle ou la sélectionner dans la conversation.
- Tester avec des questions dont la réponse est clairement présente dans les documents.

Étape 9 — Comparer sans RAG / avec RAG

Question	Sans RAG	Avec RAG	Meilleure réponse ?	Hallucination ?

Pour chaque réponse, évaluez : exactitude, utilité, niveau de détail, inventions éventuelles et bonne utilisation des documents.

7. Phase 3 — Dockerisation et déploiement sur ESXi

Objectif : transformer le prototype local en un service accessible sur le réseau TECHCORP. La VM héberge Ollama et Open WebUI dans Docker.

Étape 10 — Préparer la VM Ubuntu

```
hostnamectl
ip addr
free -h
df -h
docker --version
docker compose version
```

Recommandation de laboratoire : au moins 4 vCPU, 8 à 16 Go de RAM selon le modèle, et suffisamment d'espace disque pour le modèle et les documents.

Étape 11 — Créer le projet Docker Compose

```
sudo mkdir -p /opt/techcorp-ai/workspace
cd /opt/techcorp-ai
sudo nano compose.yml
```

```
services:
  ollama:
    image: ollama/ollama:latest
    container_name: ollama
    restart: unless-stopped
    volumes:
      - ollama_data:/root/.ollama
      - ./workspace:/workspace
    ports:
      - "11434:11434"

  open-webui:
    image: ghcr.io/open-webui/open-webui:main
    container_name: open-webui
    restart: unless-stopped
    depends_on:
      - ollama
    environment:
      - OLLAMA_BASE_URL=http://ollama:11434
    volumes:
      - open_webui_data:/app/backend/data
    ports:
      - "3000:8080"

volumes:
  ollama_data:
  open_webui_data:
```

```
docker compose up -d
docker compose ps
docker compose logs -f
```


Étape 12 — Charger votre modèle personnalisé

Copiez le Modelfile et, si nécessaire, l'adapter dans /opt/techcorp-ai/workspace.

```
docker exec -it ollama ollama pull qwen2.5:7b
docker exec -it ollama sh
cd /workspace
ollama create votre-modele -f Modelfile
ollama list
exit
```

Étape 13 — Recréer ou importer le RAG

13. Accéder à http://IP_DE_LA_VM:3000 depuis un autre poste.
14. Vérifier que votre modèle personnalisé est visible.
15. Créer/importer la collection de connaissances.
16. Ajouter les documents RAG.
17. Rejouer les tests réalisés en local.

8. Plan de recette

Test	Action	Résultat attendu	OK / KO
Accès Web	Ouvrir Open WebUI depuis un autre poste	La page se charge via l'IP de la VM	
Modèle	Envoyer une question générale	Le modèle répond	
Spécialité	Envoyer 3 questions métier	Réponses cohérentes avec le profil choisi	
RAG	Questionner une information présente uniquement dans un document	La réponse exploite correctement la connaissance	
Inconnu	Poser une question dont les informations manquent	L'assistant signale le manque ou demande une précision	
Persistance	Redémarrer les conteneurs	Modèle et données Open WebUI restent disponibles	

Test supplémentaire selon le profil

- Développement : expliquer une erreur de code ou une réponse HTTP, puis proposer une correction justifiée.
- Réseau : proposer une démarche de diagnostic avant de modifier une configuration.
- Système : utiliser les logs et l'état des services dans l'ordre logique.
- DevOps : expliquer un échec Docker/CI/CD et identifier la couche concernée.
- Cybersécurité : distinguer observation, hypothèse et action de remédiation.

9. Livrables à remettre

- Nom de l'assistant et profil métier choisi.
- Adresse IP de la VM et URL Open WebUI.
- Nom du modèle de base.
- Dataset de 50 à 100 exemples instruction/réponse.
- Modelfile final.
- Adapter LoRA si le fine-tuning a été réalisé et exporté avec succès.
- Liste des documents RAG.
- Fichier compose.yml.
- 5 à 8 captures couvrant modèle local, Open WebUI, personnalisation, RAG et déploiement.
- Tableau de recette complété.
- Bilan d'une page : réussites, limites, erreurs rencontrées et améliorations possibles.

Structure du dossier rendu

```

NOM_PRENOM_IA/
├── dataset/
│   └── dataset.json
├── model/
│   ├── Modelfile
│   └── adapter/    # si utilisé
├── rag/
│   └── documents/
├── deploy/
│   └── compose.yml
├── captures/
└── bilan.pdf ou bilan.docx
  
```

10. Barème proposé /20

Évaluation	Points	Critères
Installation locale	3	Ollama + Open WebUI opérationnels, compréhension de la chaîne
Dataset et spécialisation	4	50 à 100 exemples cohérents et adaptés au profil
Modelfile / fine-tuning	3	rôle système pertinent, paramètres expliqués, tentative LoRA documentée
RAG	4	documents pertinents, comparaison sans/avec RAG, amélioration démontrée
Déploiement	4	Docker Compose, VM ESXi, accès depuis un autre poste, persistance
Qualité du rendu	2	tests, captures, organisation des fichiers, analyse des limites

11. Commandes de diagnostic

Ollama local	ollama list ollama ps
Port Ollama	curl http://localhost:11434/api/tags
Conteneurs	docker ps docker compose ps
Logs	docker compose logs -f
Réseau VM	ip addr ss -lntp
Ressources	free -h df -h
Open WebUI	curl http://localhost:3000

Pannes fréquentes

Symptôme	Cause probable	Vérification
Open WebUI ne voit pas Ollama	mauvaise URL Ollama	vérifier OLLAMA_BASE_URL et les logs
Modèle très lent	RAM/CPU insuffisants	free -h, modèle plus petit
Port 3000 inaccessible	pare-feu ou publication de port	ss -lntp, règles firewall
Réponse RAG incorrecte	document absent, mal indexé ou question ambiguë	tester une question avec réponse exacte dans le document
Erreur de code mal expliquée	contexte de code insuffisant	fournir traceback, version, extrait minimal, entrée attendue

12. Questions de soutenance

- Quelle différence entre Ollama et Open WebUI ?
- Pourquoi le RAG est-il adapté à une documentation qui change souvent ?
- Quelle différence entre un prompt SYSTEM et un fine-tuning ?
- Quel effet peut avoir une température trop élevée pour un assistant technique ?
- Pourquoi conserver les volumes Docker ?
- Expliquez la chaîne réseau complète lorsqu'un utilisateur envoie une question depuis son navigateur.
- Profil Développement : quelle différence entre une API, FastAPI et un endpoint ?
- Profil Réseau/Système : à quel endroit de la chaîne cherchez-vous en premier si Open WebUI est accessible mais aucune réponse n'arrive ?
- Quelle limite principale avez-vous observée avec votre modèle local ?

Checklist finale

- ☐ Ollama est installé et le modèle de base fonctionne.
- ☐ Open WebUI communique avec Ollama.
- ☐ Le profil métier choisi est clairement indiqué.
- ☐ Le dataset contient au moins 50 exemples relus.
- ☐ Les exemples sont adaptés au métier, y compris pour les développeurs si ce profil est choisi.
- ☐ Le Modelfile est documenté et le modèle personnalisé est créé.
- ☐ Le fine-tuning est réalisé ou la tentative est documentée.
- ☐ La base RAG contient plusieurs documents et améliore certaines réponses.
- ☐ Le service est déployé sur la VM ESXi avec Docker Compose.
- ☐ Le service est accessible depuis un autre poste.

- ☐ Les captures, fichiers et bilan sont présents.